

Reviewer Pack — Minimal Order of Hodge Classes and Circuit Satisfiability Th...

Entry f47529d81848 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 15:37:24 UTC

Minimal Order of Hodge Classes and Circuit Satisfiability Threshold

Entry ID: f47529d81848 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 15:37:24 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Satisfiability)

Statement:

For every d -regular graph G , the minimal order of Hodge classes ($m_h(G)$) associated with its vertices is linearly correlated with its circuit satisfiability threshold, such that $m_h(G) = \Theta(\text{th}(G))$, where $\text{th}(G)$ is the minimum number of AND and OR gates required to construct a Boolean circuit satisfying all clauses of G .

Rationale (proposer's reasoning):

Hodge theory provides a rich algebraic structure for studying geometric objects, including graphs. By examining the minimal order of Hodge classes in a graph's clause representation, we may uncover hidden structures that could reveal properties about the complexity of satisfying those clauses. This connection has not been extensively explored in complexity theory.

Taxonomy category: algorithmic_lower_bounds (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 905978a03fe2fd00

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal order of Hodge classes ($m_h(G)$) is linearly correlated with the satisfiability threshold ($\text{th}(G)$) such that $m_h(G) = \Theta(\text{th}(G))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Theory" AND "Boolean Circuit Complexity" AND "circuit satisfiability threshold" - "minimal order of Hodge classes" AND "d-regular graph" AND "circuit satisfiability" - "algebraic geometry" AND "Boolean circuit gates" AND "Hodge theory"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = [[] for _ in range(n)]
    edges_added = set()

    while len(edges_added) < (n * d) // 2:
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)

        if u == v or (u, v) in edges_added or (v, u) in edges_added:
            continue

        graph[u].append(v)
        graph[v].append(u)
        edges_added.add((u, v))

    return graph

def is_bipartite(graph):
    n = len(graph)
    color = [-1] * n

    def dfs(node, c):
        if color[node] != -1:
            return color[node] == c

        color[node] = c
```

```

        for neighbor in graph[node]:
            if not dfs(neighbor, 1 - c):
                return False

        return True

    for i in range(n):
        if color[i] == -1 and not dfs(i, 0):
            return False

    return True

def circuit_satisfiability_threshold(graph):
    n = len(graph)

    def is_satisfiable(assignment):
        for u in range(n):
            clause = [graph[u][i] ^ assignment[graph[u][i]] for i in range(len(graph[u]))]
            if all(not c for c in clause):
                return False
        return True

    max_clauses = 0
    for _ in range(100): # Try 100 random assignments
        assignment = [random.randint(0, 1) for _ in range(n)]
        if is_satisfiable(assignment):
            max_clauses += 1

    return max_clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        graph = generate_d_regular_graph(n, 3)

        if not is_bipartite(graph):
            continue

        th_G = circuit_satisfiability_threshold(graph)

        # Minimal order of Hodge classes (m_h(G)) for a d-regular bipartite graph
        m_h_G = n // 2

        results.append((n, m_h_G, th_G))

    if not results:
        return {
            "metric_name": "Pearson correlation coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,

```

```

        "counterexample": "No valid bipartite graph found for  $n \leq 40$ "
    }

    n_values, m_h_Gs, th_Gs = zip(*results)
    mean_m_h_G = sum(m_h_Gs) / len(m_h_Gs)
    mean_th_G = sum(th_Gs) / len(th_Gs)

    correlation_coefficient = (sum((m - mean_m_h_G) * (t - mean_th_G) for m, t in zip(m_h_Gs, th_Gs)) /
                              math.sqrt(sum((m - mean_m_h_G)**2 for m in m_h_Gs) *
                                          sum((t - mean_th_G)**2 for t in th_Gs)))

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": abs(correlation_coefficient) > 0.95, # Threshold for linear correlation
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys

    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 37)) # Default to first 30 primes

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all("metric_value" in r and not math.isnan(r["metric_value"]) for r in results):
        mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_metric_value} std=NA support_fraction={support_fraction}")
        else:
            print(f"RESULT: FALSIFIED counterexample=\"Not enough support\" first_failing_seed=NA")
    else:
        print("RESULT: INCONCLUSIVE Some trials had NaN metric values")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61623464.py", line 131, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61623464.py", line 84, in run_trial

```

```
graph = generate_d_regular_graph(n, 3)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61623464.py", line 20, in generate_d_re
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide a Pearson correlation coefficient and p-value for the analysis. | next: Re-run the test with proper input parameters to ensure that the graph size is a multiple of the degree, and then analyze the results to determine if the conjecture holds.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13742
2	preregistration	ollama_remote	glm4:latest	0	0	9460
3	novelty	ollama_remote	glm4:latest	0	0	12342
4	novelty	ollama_remote	glm4:latest	0	0	8898
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44228
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14624
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11367
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13716
9	judge	ollama_remote	glm4:latest	0	0	12475

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140853 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f47529d81848.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f47529d81848.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f47529d81848.tar.gz (if generated)